

MACHINE LEARNING BOOTCAMP · DAY 2 OF 5

Data, EDA & Supervised Learning

From raw data to your first real predictions



Soyinka Enoch Oluwatimilehin · 29/09/2026

Quick recap of Day 1

1

ML learns the rules from **data and answers**

2

Training loop: **guess** → **error** → **adjust** → **repeat**

3

Supervised: **regression** (numbers) and **classification** (categories)

Warm-up: predicting tomorrow's rainfall in mm — regression or classification?

Today's plan — 3 hours

Time	Block	What we'll do
0:00 – 0:10	Welcome	Day 1 recap, today's plan
0:10 – 0:40	Data tools	NumPy and Pandas
0:40 – 1:10	EDA	Exploring the Titanic with charts
1:10 – 1:35	Cleaning	Missing values, encoding, features
1:35 – 1:45	Break	Stretch, questions
1:45 – 2:20	Regression	Predicting house prices
2:20 – 2:55	Classification	Predicting survival, metrics
2:55 – 3:00	Wrap-up	Recap, homework

PART 1

NumPy & Pandas

The two libraries behind almost every ML notebook.

NumPy: fast maths on arrays

Plain Python list

```
prices = [100, 250, 80]  
doubled = [p * 2 for p in prices]
```

NumPy array

```
prices = np.array([100, 250, 80])  
doubled = prices * 2
```

Fast

Written in C, so big calculations run quickly

Less code

Maths on a whole array at once, no loops

Foundation

Pandas, scikit-learn and TensorFlow build on it

NumPy essentials

1 · Create arrays

```
a = np.array([3, 7, 1, 9])  
z = np.zeros((2, 3)) # 2 rows, 3 cols  
r = np.arange(0, 10, 2)  
# [0 2 4 6 8]
```

2 · Shape & indexing

```
g = np.array([[1, 2, 3],  
              [4, 5, 6]])  
g.shape # (2, 3)  
g[1, 2] # 6
```

3 · Maths on every element

```
c = np.array([30, 25, 35])  
f = c * 9/5 + 32  
# [86. 77. 95.]
```

4 · Statistics & filtering

```
a.mean() # 5.0  
a.max() # 9  
a.std() # 3.16  
a[a > 5] # [7 9]
```

Pandas: tables in Python

A **DataFrame** is a table, like an Excel sheet. Each column is a **Series**.

Command	What it tells you
<code>pd.read_csv("file.csv")</code>	Loads a CSV file into a DataFrame
<code>df.head()</code>	The first 5 rows
<code>df.shape</code>	(rows, columns)
<code>df.info()</code>	Column types and missing values
<code>df.describe()</code>	Mean, min, max of number columns

Pandas essentials

1 · Pick columns

```
df["age"] # one column  
df[["sex", "age"]] # two
```

2 · Filter rows

```
kids = df[df["age"] < 18]  
women = df[df["sex"] == "female"]
```

3 · Group & summarise

```
df.groupby("sex")["survived"].mean()  
# female 0.74  
# male    0.19
```

4 · Count & check

```
df["embarked"].value_counts()  
df.isna().sum() # missing  
df.sort_values("fare")
```

PART 2

Exploratory Data Analysis

Get to know your data before you model it.

EDA: interview your data

How big is it?

`df.shape`

What types?

`df.info()`

What's missing?

`df.isna().sum()`

How is it spread?

Histogram, box plot

What's related?

Scatter plot, heatmap

Anything weird?

Outliers, typos, odd values

EDA finds the garbage before it goes in.

Meet the Titanic dataset

891

passengers (rows)

38%

survived

```
sns.load_dataset(  
    "titanic")
```

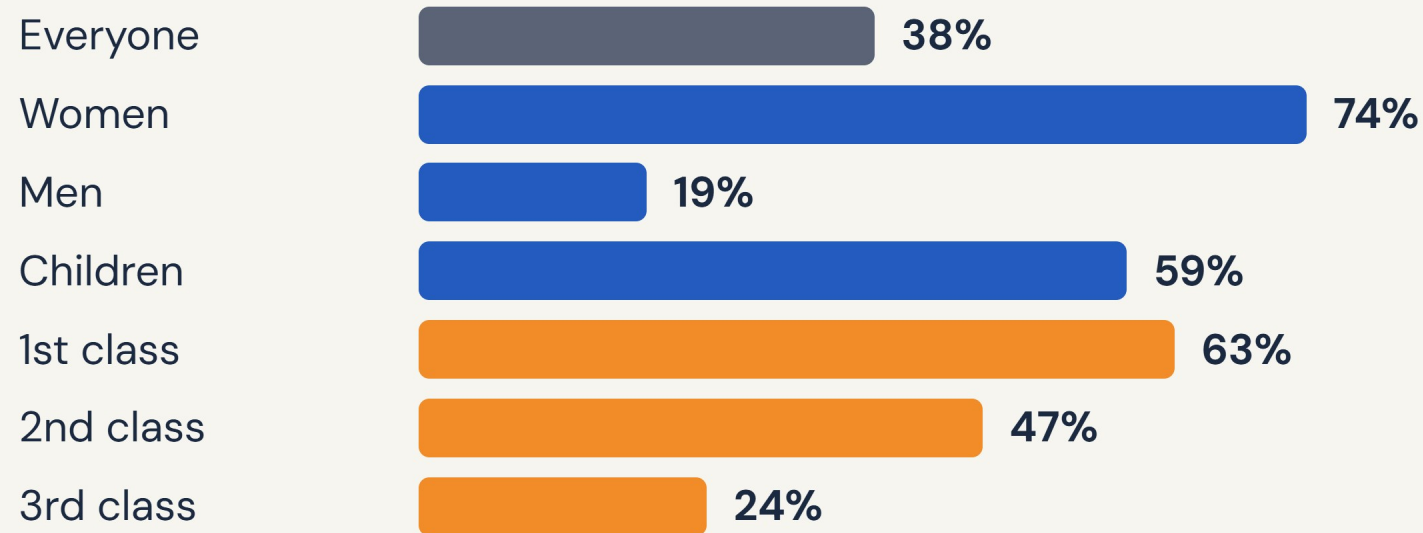
Column	Meaning
survived	0 = died, 1 = survived (the label)
pclass	Ticket class: 1st, 2nd or 3rd
sex	male / female
age	Age in years (177 missing)
sibsp	Siblings + spouses aboard
parch	Parents + children aboard
fare	Ticket price
embarked	Port: C, Q or S

Which plot for which question?

Question	Plot	Seaborn
How is one number spread?	Histogram	sns.histplot
Are there outliers?	Box plot	sns.boxplot
How many in each group?	Count plot	sns.countplot
Do two numbers move together?	Scatter plot	sns.scatterplot
Which columns are related?	Heatmap	sns.heatmap

All of them take the same inputs: **data=df, x="column", hue="group"**

Who survived the Titanic?



Sex and class are strong signals — great features for our model.

PART 3

Cleaning & feature engineering

Turning raw data into model-ready inputs.

Handling missing values

age: 177 missing (20%)

deck: 688 missing (77%)

embarked: 2 missing

Drop the column

When most values are missing

Titanic: deck

Fill it in

Median for numbers, most common value for categories

Titanic: age → 28, **embarked** → S

Drop the rows

When only a few rows are affected

Careful: you lose data

```
df["age"] = df["age"].fillna(df["age"].median())
```

Encoding: words into numbers

Binary encoding

Two categories → 0 and 1

sex	sex (encoded)
male	0
female	1
female	1

One-hot encoding

No order → one 0/1 column each

embarked	C	Q	S
S	0	0	1
C	1	0	0
Q	0	1	0

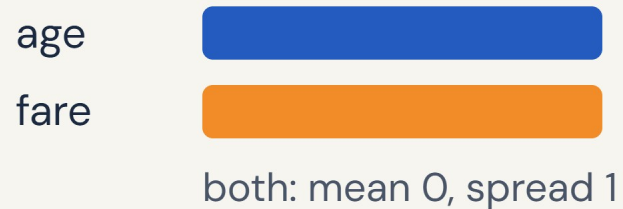
Why not S = 1, C = 2, Q = 3? The model would think Q is "bigger" than C — nonsense.

Scaling: one ruler for all features

Before



After standardising



StandardScaler

Mean 0, spread (std) 1

$$z = (x - \text{mean}) \div \text{std}$$

MinMaxScaler

Squeezes values into 0 to 1

$$(x - \text{min}) \div (\text{max} - \text{min})$$

KNN, logistic regression and neural networks need scaling. **Decision trees don't.**

Feature engineering: better inputs

Titanic

family size = $\text{sibsp} + \text{parch} + 1$

is alone = 1 if family size is 1

Alone: **30%** survived · With family: **51%**

Housing

rooms per household = $\text{rooms} \div \text{households}$

bedrooms per room = $\text{bedrooms} \div \text{rooms}$

Link to price: bedrooms **0.05** → bedrooms per room **-0.23**

Good features often beat fancy models.

Train/test split & data leakage

Training set · 80%

The model learns from these rows

Test · 20%

Final exam

Golden rules

1. Split before fitting scalers or models
2. Fit on train, only transform test
3. Keep the test set locked until the end

Data leakage

When the answer sneaks into the features.

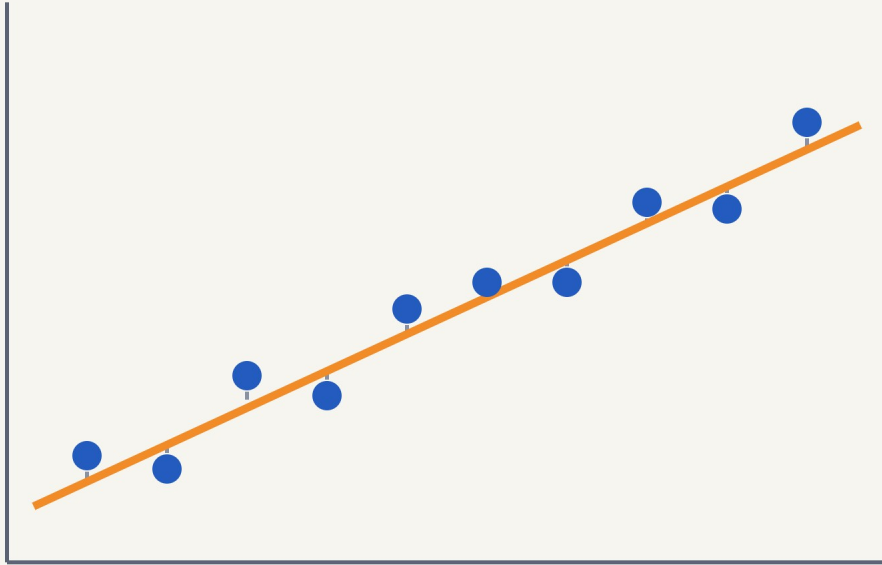
Titanic's **"alive"** column is just "survived" in words. Keep it and you get a perfect score that means nothing.

PART 4

Regression

Predicting numbers: California house prices.

Linear regression: the best line



Income (x) vs house price (y) · dashed = errors

$$\text{price} = w \times \text{income} + b$$

w (slope): how much price rises per unit of income

b (intercept): the starting value

Best line = smallest total squared error

Our data: price $\approx$ **\$41,934** $\times$ income + **\$44,460**

How good is a regression model?

MAE

Mean Absolute Error

The average size of the mistakes

Ours: off by **\$50.9k** on average

RMSE

Root Mean Squared Error

Like MAE, but big mistakes count extra

Ours: **\$72.7k**

R²

R-squared

1 = perfect, 0 = no better than guessing the average

Ours: **0.60**

Lower MAE and RMSE is better. **Higher R² is better.**

California housing: results

20,640 districts · predict median house value · 20% held back for testing

Model	MAE	RMSE	R ²
Guess the average	\$90.6k	\$114.5k	0.00
Income only	\$63.0k	\$84.2k	0.46
Linear regression (all)	\$50.9k	\$72.7k	0.60
Random forest (Day 3)	\$32.3k	\$50.4k	0.81

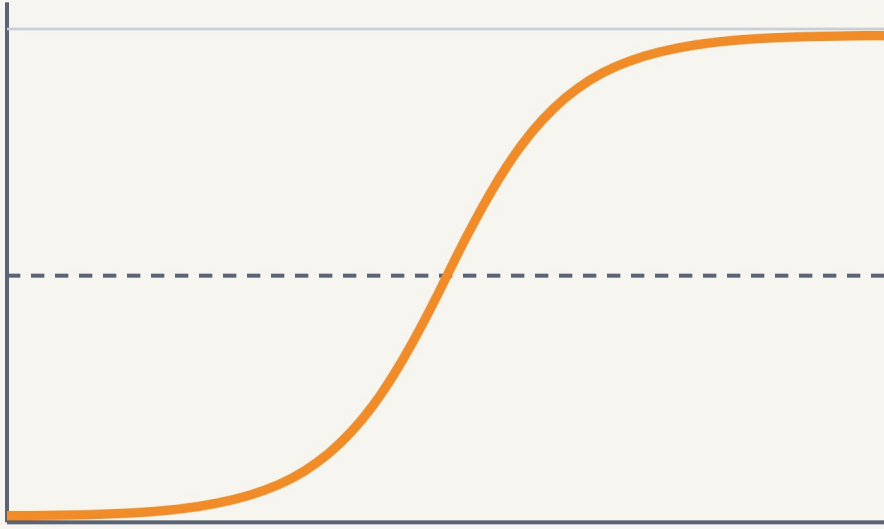
More good features → better. A smarter model → better still.

PART 5

Classification

Predicting categories: who survived the Titanic?

Logistic regression: a probability



Probability 0 → 1 · dashed line = 0.5 cut-off

Outputs a **probability** from 0 to 1

Above 0.5 → predict "**survived**"

Despite the name, it's for **classification**

What it learned on Titanic

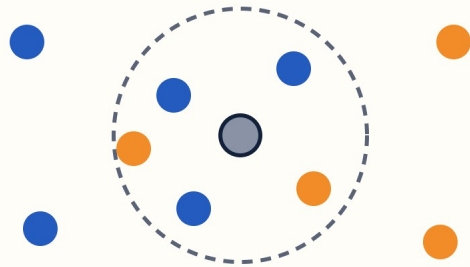
Female → much higher chance

3rd class → lower chance

Older → slightly lower chance

Two more classifiers

K-nearest neighbours



Find the **k most similar** passengers and take a vote.

$k = 5$: 3 blue, 2 orange $\rightarrow$ predict **blue**

Decision tree

A flowchart of yes/no questions, learned from data.

Female?

Yes $\rightarrow$ 1st or 2nd class?
 $\rightarrow$ likely survived

No $\rightarrow$ a child? $\rightarrow$ else
likely died

Simplified example

The confusion matrix

	Predicted: died	Predicted: survived	
Actually died	97 correct	13 false positive	True positive (47): said survived, did survive True negative (97): said died, did die False positive (13): said survived, but died
Actually survived	22 false negative	47 correct	False negative (22): said died, but survived

Accuracy isn't everything

Accuracy

80.4%

How often is it right overall?

$$(97 + 47) \div 179$$

Precision

78.3%

When it says "survived", how often is it right?

$$47 \div (47 + 13)$$

Recall

68.1%

Of the real survivors, how many did it find?

$$47 \div (47 + 22)$$

F1 score

72.9%

One number balancing precision and recall

$$2 \times P \times R \div (P + R)$$

Titanic: comparing classifiers

Model	Accuracy	Precision	Recall	F1
Always say "died"	61.5%	—	0%	—
Logistic regression	80.4%	78.3%	68.1%	72.9%
KNN (k = 5)	81.6%	79.0%	71.0%	74.8%
Decision tree (depth 4)	78.8%	84.4%	55.1%	66.7%

Doing nothing already scores 61.5% — always compare with a baseline.

No single winner: the tree is most precise, KNN finds the most survivors.

Warning sign: overfitting

Decision tree, no limit



Decision tree, max depth 4



WHAT HAPPENED?

The unlimited tree **memorised** the training passengers — like memorising past questions word for word.

Big gap between training and test = overfitting. Fixes come on Day 3.

The scikit-learn recipe

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler().fit(X_train)
X_train, X_test = scaler.transform(X_train), scaler.transform(X_test)

model = LogisticRegression().fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))
```

1 · Split



2 · Scale (fit on train)



3 · Fit



4 · Evaluate

Swap in any other model — **the steps stay the same.**

Recap: what we covered today

- 1 NumPy for arrays, Pandas for tables
- 2 EDA first: shape, types, missing values, plots
- 3 Clean: fill or drop, encode, scale, new features
- 4 Regression: MAE, RMSE, R^2 · Classification: accuracy, precision, recall
- 5 Always beat a baseline, on unseen data

BEFORE DAY 3

- Finish the 3 Day 2 notebooks
- Add one new Titanic feature — does accuracy improve?
- Try KNN with $k = 1, 5, 15$

Tomorrow: random forests, SVM, overfitting fixes, clustering, PCA

Questions?

Tomorrow: more models, overfitting and unsupervised learning.

Soyinka Enoch Oluwatimilehin · 08144801748